

Module request flow

File-by-file walkthrough: which file runs first, what logic it owns, and how a request travels from the browser to Postgres and back.

Stack: Next.js `apps/web` (:3000) → Nest gateway `apps/api-gateway` (:3001) → Nest domain API `apps/api` (:3002) → Postgres (:5434)

Generated from the current codebase. Read this in order — later modules reuse the same hop.

Contents

1. How to read this document
2. Big picture and startup order
3. The shared hop (every API call)
4. Auth (register, login, me, logout, address)
5. Browse restaurants and menus
6. Cart (one restaurant invariant)
7. Place order + payment
8. Kitchen status updates
9. Customer / admin order lists
10. Admin creates a restaurant + staff account
11. Menu CRUD and soft-delete
12. Image upload (Cloudinary)
13. Roles, layouts, and page map
14. Mock mode (no backend)
15. Cheat sheet: who owns what

1. How to read this document

Every user action is a chain of files. The numbered lists below are that chain in order. The first file is what the user touches. The last file is Postgres (or Cloudinary / Razorpay).

Two kinds of traffic

Pages stay on Next.js (HTML/React). They never talk to :3002.

API calls always go `/api/...` on :3000, which Next rewrites to the gateway.

Role in JWT	Who they are	Home after login
user	Customer	/restaurants
staff	Restaurant kitchen	/restaurant/dashboard
admin	Platform operator	/admin

Registration always creates `user`. Staff accounts come from seed or from admin creating a restaurant. Users live only in the gateway DB. Domain tables store `userId` — never email/password.

2. Big picture and startup order

```
Browser  :3000  apps/web          (Next pages + axios)
      |
      | /api/* rewrite
      v
Gateway  :3001  apps/api-gateway (cookie JWT, /auth, /health, /docs)
      |
      | proxy + Cookie → Authorization: Bearer
      v
Domain   :3002  apps/api          (restaurants, cart, orders, payments, uploads)
      |
      v
Postgres :5434  docker/docker-compose.yml
```

What starts first when you run `pnpm dev`

- 1 Root `package.json` runs Turbo in parallel for three apps.
- 2 `apps/api-gateway/src/load-env.ts` then `apps/api-gateway/src/main.ts` — Nest factory, cookie parser, CORS, proxy middleware, Swagger, listen on :3001. Module graph: `apps/api-gateway/src/app.module.ts`.
- 3 `apps/api/src/load-env.ts` then `apps/api/src/main.ts` — Nest factory, pipes/filters/interceptors, Swagger, listen on :3002. Module graph: `apps/api/src/app.module.ts`.
- 4 Next starts `apps/web`. First HTML shell is `apps/web/app/layout.tsx` (font, theme script, CSS).
- 5 `apps/web/components/providers/app-providers.tsx` wraps the tree: Redux → TanStack Query → theme → `AuthProvider` → `overlays/toasts`.

- 6 On every page load, `apps/web/components/auth/auth-provider.tsx` calls `GET /auth/me` so the header knows who is logged in.

Env files (read at process start)

App	Loader	Typical file
Gateway	<code>apps/api-gateway/src/load-env.ts</code> + <code>libs/env</code>	<code>apps/api-gateway/.env</code>
Domain API	<code>apps/api/src/load-env.ts</code>	<code>apps/api/.env</code>
Web	Next reads <code>NEXT_PUBLIC_*</code>	<code>apps/web/.env.local</code>
Postgres URL / JWT	Shared secrets	Root <code>.env</code> + both Nest apps

3. The shared hop (every API call)

Almost every feature below uses this exact pipeline. Learn it once. Example: customer adds a burger → `POST /api/cart/items`.

A. Browser → Next

- 1 Page or hook calls `foodApi.addToCart(...)` from `apps/web/lib/food-api.ts`. If `NEXT_PUBLIC_USE MOCK=true`, this is fake data in `apps/web/lib/mock/handlers.ts` and the request never leaves the browser.
- 2 Real mode uses `apps/web/lib/api/food-delivery.ts` which calls axios `ApiClient` from `apps/web/lib/api.ts`.
- 3 Base URL comes from `apps/web/lib/api-base-url.ts` → default `/api` (same origin, port 3000). Cookies are sent (`withCredentials: true`) via `libs/api-client`.
- 4 Request interceptor in `apps/web/lib/api.ts` may start the full-screen loading overlay (`apps/web/lib/api-loading.ts`). Cart quantity changes skip it.
- 5 `apps/web/next.config.ts` rewrite: `/api/:path*` → `http://localhost:3001/:path*`. The browser still thinks it hit Next.

B. Gateway (:3001) — cookie or proxy

- 1 Express middleware in `apps/api-gateway/src/main.ts`: compression, security headers (`libs/http/src/middleware/security-headers.ts`), cookie-parser, request id (`libs/http/src/middleware/request-id.ts`).
- 2 `apps/api-gateway/src/common/proxy-hop.ts` → `isGatewayOwnedPath`. Paths starting with `/auth`, `/health`, `/docs` stay on the gateway. Everything else is proxied to `API_UPSTREAM_URL` (default :3002).

3 On proxy: same file `applyAuthCookieToProxyRequest` copies cookie `access_token` into `Authorization: Bearer ...` and strips the Cookie header so the domain API never sees browser cookies.

4 If the path is gateway-owned, Nest handles it. Global guards from `apps/api-gateway/src/app.module.ts`: `Throttler` → `JwtAuthGuard` → `RolesGuard` (implementations in `libs/http/src/auth/guards.ts`, re-exported from `apps/api-gateway/src/common/auth/index.ts`).

C. Domain API (:3002) — Bearer JWT + business rules

1 Same Nest pipeline in `apps/api/src/main.ts`: request id, security headers, `ValidationPipe`, `AllExceptionsFilter`, `ResponseEnvelopeInterceptor`.

2 `JwtAuthGuard` + `RolesGuard` from `apps/api/src/app.module.ts`. JWT is read from the Bearer header by `apps/api/src/modules/auth/strategies/jwt.strategy.ts` (does *not* hit the users table — it trusts claims: id, email, role).

3 Controller method runs (route + `@Roles` + DTO).

4 Service contains the real rules (400/403, transactions).

5 TypeORM entity maps to Postgres. Success body is wrapped as `{ data: ... }` by `libs/http/src/interceptors/response-envelope.interceptor.ts`. Errors stay unwrapped: `{ statusCode, error, message }` via `libs/http/src/filters/all-exceptions.filter.ts`.

D. Response back to the UI

1 Axios in `libs/api-client` unwraps `{ data }` with `unwrapData()`. 401 (except login/me/register) sends the browser to `/login` via `apps/web/lib/api.ts`.

2 TanStack Query hook in `apps/web/lib/hooks/food-delivery.ts` puts the result in cache using keys from `apps/web/lib/query-keys.ts`.

3 The page re-renders. Toasts come from `apps/web/lib/toast.ts` / `use-toast-query-error.ts`.

Gateway JWT strategy (`apps/api-gateway/src/modules/auth/strategies/jwt.strategy.ts`) **loads the user from the database**. Domain JWT strategy only reads token claims. Both share `JWT_SECRET`.

4. Auth module

Auth never goes to `apps/api`. Gateway owns users and cookies.

4.1 Login — POST `/api/auth/login`

1 UI: `apps/web/app/(auth)/login/page.tsx` validates with `parseLogin` in `apps/web/lib/validation/food-delivery.ts`.

- 2 Calls `login()` on `apps/web/components/auth/auth-provider.tsx`.
- 3 Provider calls `authApi.login` from `apps/web/lib/api.ts` (built by `createAuthApi` in `libs/api-client`).
- 4 Next rewrite → gateway. Path is `/auth/login` so it is **not proxied**.
- 5 `@Public()` + throttle on `apps/api-gateway/src/modules/auth/auth.controller.ts`. Body DTO: `apps/api-gateway/src/modules/auth/dto/auth.dto.ts`.
- 6 `apps/api-gateway/src/modules/auth/auth.service.ts` `validateUser` : load user by email via `apps/api-gateway/src/modules/users/users.service.ts`, `bcrypt-compare` password (dummy hash if email missing, to avoid timing leaks).
- 7 Same service `issueSession` : signs JWT (sub, email, role) and sets httpOnly cookie `access_token`. Returns public user (`dto/public-user.dto.ts` shape: id, email, name, role, deliveryAddress).
- 8 Envelope wraps the user. Browser stores the cookie automatically. Login page uses `apps/web/lib/auth-routes.ts` `homePathForRole` to send staff → dashboard, admin → /admin, customer → /restaurants.

4.2 Register — `POST /api/auth/register`

- 1 `apps/web/app/(auth)/register/page.tsx` → `AuthProvider` `register()`.
- 2 Controller same file as login. Role is hard-coded to `user`.
- 3 Service hashes password (`bcrypt 12`), `UsersService.create`, unique email (409), then same cookie as login.
- 4 Table: `apps/api-gateway/src/modules/users/user.entity.ts`.

4.3 Session check — `GET /api/auth/me`

- 1 `AuthProvider` `refresh()` on app start.
- 2 Not `@Public` — `JwtAuthGuard` requires cookie.
- 3 Passport extracts cookie in gateway JWT strategy, loads user by id from DB, attaches public user.
- 4 Controller `me()` just returns that user. No extra service call.

4.4 Logout — `POST /api/auth/logout`

`AuthProvider` → controller (public) → `AuthService.logout` clears cookie. Provider also clears TanStack food cache (`apps/web/lib/food-cache.ts`) and Redux filters (`apps/web/lib/store/index.ts`).

4.5 Save delivery address — `PATCH /api/auth/address`

- 1 Cart page `apps/web/app/cart/page.tsx` can save address via `AuthProvider` `saveAddress` .
- 2 DTO: `save-delivery-address.dto.ts` .
- 3 `UserService` updates `delivery_address` (migration `apps/api-gateway/src/database/migrations/1787000000000-AddUserDeliveryAddress.ts`).

5. Browse restaurants and menus

Public routes — JWT optional. No login required.

5.1 Restaurant list — `GET /api/restaurants?cuisine=&q=&page=`

- 1 Page `apps/web/app/restaurants/page.tsx` . Filter chips/search write *draft* values into `Redux` (`apps/web/lib/store/index.ts`) via `apps/web/components/food/restaurant-filters.tsx` , then apply them. Debounce: `apps/web/lib/hooks/use-debounced-value.ts` .
- 2 Hook `useRestaurants` in `apps/web/lib/hooks/food-delivery.ts` reads *applied* `Redux` filters and calls `foodApi.listRestaurants` .
- 3 Gateway proxies (not `/auth`). Domain `apps/api/src/modules/restaurants/restaurants.controller.ts` `@Public()` `list()` .
- 4 Query DTO: `dto/list-restaurants-query.dto.ts` (cuisine, q, page, limit).
- 5 `restaurants.service.ts` `list()` : `TypeORM` query on `restaurant.entity.ts` , `ILIKE` filters, pagination helper `apps/api/src/common/pagination.ts` .
- 6 Cards: `apps/web/components/food/restaurant-card.tsx` , diet badge `diet-badge.tsx` , image `food-image.tsx` .

5.2 Restaurant detail + menu — `GET /api/restaurants/:id` then `GET /api/menu-items?restaurantId=`

- 1 Page `apps/web/app/restaurants/[id]/page.tsx` .
- 2 Hooks `useRestaurant(id)` + `useMenuItems(id)` .
- 3 Menu list: `menu-items.controller.ts` `public list()` → `menu-items.service.ts` . Soft-deleted rows (`deletedAt`) are hidden unless `includeDeleted=true` and caller is staff/admin of that restaurant.
- 4 UI cards: `apps/web/components/food/menu-item-card.tsx` .

6. Cart (one restaurant per cart)

Requires login as `user` (or admin). Layout gate: page wraps `RequireRole`.

6.1 Load cart — `GET /api/cart`

- 1 Hook `useCart` only runs when auth user role is user/admin.
- 2 `cart.controller.ts` `@Roles('user','admin')`.
- 3 `cart.service.ts` `getCart`: join `cart_items` → `menu_items` → `restaurants`, skip deleted menu items, return lines + `restaurantId/name`.
- 4 Entity: `apps/api/src/modules/cart/cart-item.entity.ts`. DB unique: `(user_id, menu_item_id)` in init migration.

6.2 Add item — `POST /api/cart/items`

- 1 Menu page calls `useAddToCart`. If the cart already belongs to another restaurant, UI first opens `apps/web/components/food/cart-switch-dialog.tsx` (confirm → clear cart → add).
- 2 DTO: `dto/add-cart-item.dto.ts`.
- 3 Service: load menu item; 404 if missing/deleted; if existing cart restaurant ≠ this item's restaurant → **400** "Cart already has items from another restaurant"; else bump quantity or insert row.

6.3 Change qty / clear

Action	HTTP	Files
Qty / remove	PATCH <code>/cart/items/:id</code>	Hook <code>useUpdateCartItem</code> (optimistic UI) → <code>cart.service.ts</code> <code>updateItem</code> (qty 0 deletes)
Empty cart	DELETE <code>/cart</code>	<code>useClearCart</code> → <code>clearCart</code>

Checkout UI: `apps/web/app/cart/page.tsx`. Price preview uses `apps/web/lib/pricing.ts` (mirrors backend `apps/api/src/common/pricing.ts`).

7. Place order + payment

This is the required transaction plus Stretch payments. Kitchen never sees unpaid orders.

7.1 Place order — `POST /api/orders`

- 1 Cart page validates address with `parsePlaceOrder`, then `usePlaceOrder` → `foodApi.placeOrder`.
- 2 Controller `apps/api/src/modules/orders/orders.controller.ts` `@Roles('user','admin')`. DTO: `dto/place-order.dto.ts`.

- 3 `orders.service.ts` `placeOrder` opens a **TypeORM transaction**:
 - Load cart + menu + restaurant (including soft-deleted, to reject them).
 - Empty cart → 400. Deleted item → 400. Mixed restaurants → 400.
 - `calculatePricing(subtotal)` : ₹40 delivery (free over ₹500), 5% platform fee, 5% tax.
 - Insert `Order` status `placed` , `paymentStatus` `pending` .
 - Insert `OrderLine` snapshots (name + unit price).
 - Insert first `DeliveryStatus` row = `placed`.
 - Delete all cart rows for this user.

- 4 Entities: `order.entity.ts` , `order-line.entity.ts` , `delivery-status.entity.ts` .

- 5 Cart is now empty. Order exists but is hidden from lists until paid (`payment_status = paid` filter in `list` / `getById`).

7.2 Create checkout — `POST /api/orders/:orderId/payments/create`

- 1 Same cart page immediately calls `useCreatePaymentCheckout` .
- 2 `payments.controller.ts` → `payments.service.ts` `createCheckout` . Only the order owner (or admin) may pay.
- 3 If Razorpay keys exist (`apps/api/src/config/razorpay.config.ts`), `razorpay.client.ts` creates a real Razorpay order (amount in paise). Otherwise a mock `order_mock_*` id is stored.
- 4 Payment row: `payment.entity.ts` .

7.3 Pay in the browser

- 1 Real: `apps/web/lib/razorpay.ts` loads `checkout.js` and opens Razorpay. Mock: `apps/web/components/food/mock-payment-dialog.tsx` .
- 2 On success, `useVerifyPayment` → `POST /orders/:id/payments/verify` with order id, payment id, signature.
- 3 `payments.service.ts` `verifyPayment` : HMAC check when keys exist; mock signature accepted in mock mode; sets payment paid + `order.paymentStatus = paid`.
- 4 Query cache invalidated. User is sent to `/orders/:id` (`apps/web/app/orders/[id]/page.tsx`).

Pricing is computed once at place-order time and stored on the order. Lines are snapshots, so later menu price edits do not change old receipts.

8. Kitchen status updates

Allowed path (enforced in service, not only UI):

placed → preparing | cancelled
preparing → out_for_delivery | cancelled
out_for_delivery → delivered
delivered / cancelled → stop (no reverse, no skip)

8.1 Restaurant dashboard

- 1 Layout `apps/web/app/restaurant/layout.tsx` wraps `RequireRole roles=['staff','admin']` (`apps/web/components/auth/require-role.tsx`). Logged out → redirect `/login?returnTo=...` . Wrong role → `access-denied.tsx` .
- 2 Page `apps/web/app/restaurant/dashboard/page.tsx` . Hook `useOrders('restaurant')` → `GET /orders?scope=restaurant` .
- 3 Service list: staff can only see orders for the restaurant they own (`RestaurantsService.findStaffRestaurant`). Admin with this scope sees all kitchens. Unpaid orders are excluded.
- 4 Buttons: `apps/web/components/food/status-action-buttons.tsx` only offer the next legal statuses from the same map the API uses.
- 5 Mutation `useUpdateOrderStatus` → `PATCH /orders/:id/status` .
- 6 Controller `@Roles('staff','admin')` . DTO `dto/update-order-status.dto.ts` .
- 7 `orders.service.ts` `updateStatus` :
 - Load order. Not owner → 403.
 - Not paid → 400.
 - `canMoveStatus` in `order-status.ts` — skip/back → 400.
 - Transaction: set `Order.status` ; on first preparing set `estimatedMinutes = 30` ; overwrite the single `DeliveryStatus` row (unique per order).

Timeline UI: `apps/web/components/food/order-timeline.tsx` draws a stepper from current status. Badge: `order-status-badge.tsx` . Receipt: `price-breakdown.tsx` .

9. Customer / admin order lists

UI	Hook / scope	API
<code>apps/web/app/orders/page.tsx</code>	<code>useOrders('mine')</code>	<code>GET /orders?scope=mine</code> — orders this user placed
<code>apps/web/app/orders/[id]/page.tsx</code>	<code>useOrder(id)</code>	<code>GET /orders/:id</code> — 403 if not owner/staff-of-that-kitchen/admin
<code>apps/web/app/admin/orders/page.tsx</code>	<code>useOrders('all')</code>	<code>GET /orders?scope=all</code> — admin only
<code>apps/web/app/admin/page.tsx</code>	same + restaurant counts	Platform overview

API also accepts `?status=placed` etc. (`list-orders-query.dto.ts`). The list pages currently bucket in the browser (All / Active / Delivered / Cancelled) instead of mounting `apps/web/components/food/order-filters.tsx` .

10. Admin creates a restaurant + staff account

- 1 Layout `apps/web/app/admin/layout.tsx` → admin only. Page `apps/web/app/admin/restaurants/page.tsx` .
- 2 Optional image via Cloudinary (section 12). Form → `useCreateRestaurant` .
- 3 `POST /restaurants` `@Roles('admin')` . DTO `dto/create-restaurant.dto.ts` .
- 4 `restaurants.service.ts` `create` starts a DB transaction. `staff-owner.provisioner.ts` inserts a `users` row with role `staff` (raw SQL into the shared Postgres; password like `HastyTasty@123`).
- 5 Restaurant row saved with `ownerUserId` = that staff id. Response includes one-time `staffLogin` { email, password }.
- 6 UI shows `apps/web/components/food/staff-credentials-modal.tsx` .
- 7 Staff later uses `GET /restaurants/mine` (hook `useMyRestaurant`) to find “their” restaurant for the menu editor.

Admin can also open `apps/web/app/admin/restaurants/[id]/page.tsx` (read-only menu). Staff/admin can `PATCH /restaurants/:id` (owner check in `assertCanManage`).

11. Menu CRUD and soft-delete

Page: `apps/web/app/restaurant/menu/page.tsx` (staff/admin).

Action	HTTP	Logic file
List including deleted	<code>GET /menu-items?</code> <code>restaurantId=&includeDeleted=true</code>	<code>menu-items.service.ts</code> <code>list</code> — staff only for own restaurant
Create	<code>POST /menu-items</code>	<code>create</code> + <code>assertCanManage</code>
Update	<code>PATCH /menu-items/:id</code>	<code>update</code>
Delete (soft)	<code>DELETE /menu-items/:id</code> → 204	<code>softDelete</code> sets <code>deletedAt</code> ; HTTP DELETE, not a hard wipe
Public get one	<code>GET /menu-items/:id</code>	404 if deleted (“no longer available”)

Entity: `apps/api/src/modules/restaurants/menu-item.entity.ts` . Frontend hooks: `useCreateMenuItem` , `useUpdateMenuItem` , `useDeleteMenuItem` . Confirm dialogs live on the menu page.

12. Image upload (Cloudinary)

Browser never sends the image through Nest. Nest only signs the upload.

- 1 UI: `apps/web/components/food/image-upload.tsx` / `image-upload-modal.tsx` on admin restaurant form or staff menu form.
- 2 `apps/web/lib/cloudinary.ts` validates type/size, then `GET /uploads/cloudinary-signature?folder=restaurants|menu-items`.
- 3 Gateway proxies. `uploads.controller.ts @Roles('staff','admin')` → `uploads.service.ts` (HMAC signature from `apps/api/src/config/cloudinary.config.ts`). Missing env → 503.
- 4 Browser POSTs the file **directly to Cloudinary** (`https://api.cloudinary.com/...` , allowed in Next CSP in `next.config.ts`).
- 5 Returned `secure_url` is saved on restaurant/menu via the normal PATCH/POST DTOs (`imageUrl` columns from migration `178800000000-AddImageUrls.ts`).

13. Roles, layouts, and page map

Shell/nav: `apps/web/components/layout/app-shell.tsx` + `apps/web/components/auth/shell-header.tsx` hide links the role cannot use. Do not rely on API 403 alone.

Route	Page file	Auth
/	<code>apps/web/app/page.tsx</code>	Public landing
/login /register	<code>app/(auth)/...</code>	Public
/restaurants	<code>app/restaurants/page.tsx</code>	Public
/restaurants/[id]	<code>app/restaurants/[id]/page.tsx</code>	Public (add-to-cart needs login)
/cart	<code>app/cart/page.tsx</code>	user (RequireRole)
/orders	<code>app/orders/page.tsx</code>	user
/orders/[id]	<code>app/orders/[id]/page.tsx</code>	Logged-in owner / kitchen / admin
/restaurant/dashboard	<code>app/restaurant/dashboard/page.tsx</code>	staff/admin via layout
/restaurant/menu	<code>app/restaurant/menu/page.tsx</code>	staff/admin via layout
/admin	<code>app/admin/page.tsx</code>	admin via layout
/admin/restaurants	<code>app/admin/restaurants/page.tsx</code>	admin
/admin/orders	<code>app/admin/orders/page.tsx</code>	admin

Health (no JWT): gateway `apps/api-gateway/src/modules/health/health.controller.ts` ; API liveness + `GET /ready` in `apps/api/src/modules/health/` . Browser can hit `/api/ready` (proxied).

14. Mock mode (no backend)

`NEXT_PUBLIC_USE MOCK=true` → `apps/web/lib/food-api.ts` exports `mockFoodApi` from `apps/web/lib/mock/handlers.ts` (in-memory restaurants, cart, orders) and auth from `apps/web/lib/mock/auth.ts` / seed-like `apps/web/lib/mock/data.ts`. Same hooks and pages. Zero gateway/API traffic.

15. Cheat sheet: who owns what

Concern	File(s)
Rewrite /api → gateway	<code>apps/web/next.config.ts</code>
Axios + loading overlay + 401 redirect	<code>apps/web/lib/api.ts</code> , <code>libs/api-client</code>
Food HTTP methods	<code>apps/web/lib/api/food-delivery.ts</code>
TanStack Query / mutations	<code>apps/web/lib/hooks/food-delivery.ts</code>
Filter drafts (RTK)	<code>apps/web/lib/store/index.ts</code>
Zod forms	<code>apps/web/lib/validation/food-delivery.ts</code>
TS shapes	<code>apps/web/lib/types/food-delivery.ts</code>
Cookie ↔ Bearer hop	<code>apps/api-gateway/src/common/proxy-hop.ts</code>
Login cookie + bcrypt	<code>apps/api-gateway/src/modules/auth/auth.service.ts</code>
Users table	<code>apps/api-gateway/src/modules/users/</code>
Global Nest guards	<code>libs/http/src/auth/guards.ts</code>
One-restaurant cart	<code>apps/api/src/modules/cart/cart.service.ts</code>
Place-order transaction	<code>apps/api/src/modules/orders/orders.service.ts</code>
Status state machine	<code>apps/api/src/modules/orders/order-status.ts</code>
Razorpay	<code>payments.service.ts</code> , <code>razorpay.client.ts</code> , <code>apps/web/lib/razorpay.ts</code>
Fees / tax	<code>apps/api/src/common/pricing.ts</code> (mirrored on web)
Staff user insert	<code>staff-owner.provisioner.ts</code>
Gateway seed users	<code>apps/api-gateway/src/database/seed.ts</code>
API seed restaurants/menu	<code>apps/api/src/database/seed.ts</code>

Happy-path story (one sentence per hop)

Customer opens `/restaurants/[id]` → Query loads restaurant + menu → Add to cart hits cart service (one restaurant) → Cart page places order in a transaction → Payments service creates Razorpay/mock checkout → Verify marks paid → Staff dashboard lists that order → PATCH status uses `order-status.ts` → Customer `/orders/[id]` shows badge + timeline.

Remember: pages render on :3000; JSON always goes /api → :3001; domain rules live in `apps/api` services, not in React. If a rule is only in the UI, it is not the real invariant.

TastyGo module request flow · fullstack-monorepo · for reading the current implementation.